MINI LINUX SHELL WITH BACKGROUND JOB MANAGEMENT

Project Team

P.Ruhitha	2520080053
M. Ananya	2520080069
CH. Madhulika	2520090056

2026-2027

Supervised by
M.Raghupathi



Department of AI&DS, CSIT

Koneru Lakshmaiah Education Foundation

Hyderabad

Certificate of Approval

The Department of AI&DS, CS&IT, Koneru Lakshmaiah Education Foundation, accepts this thesis titled Mini Linux Shell with Background Job Management, submitted by P. Ruhitha-2520080053, M. Ananya-2520080069, CH.Madhulika-2520090056, in its current form, and it is satisfying the dissertation requirements for the award of Bachelors Degree in [Program Name].

Supervisor

[Supervisor Name] Signature: _____

Acknowledgements

We express our sincere gratitude to Koneru Lakshmaiah Education Foundation for providing the academic environment and resources required for carrying out this project.

We would like to express our sincere thanks to our Supervisor, [Supervisor Name], for their valuable guidance, support, and encouragement throughout the design, implementation, and testing stages of the Mini Linux Shell with Background Job Management.

We are also grateful to the faculty members for their support and guidance during the completion of this project.

Finally, we thank all the members of our project team for their cooperation, coordination, and contribution towards the successful completion of this work.

P. Ruhitha
M. Ananya
CH. Madhulika

Abstract

A shell is the main way in which a user starts and controls programs on a Linux system, and background job management is one of its most important features. Yet the process-management work that a shell performs is hidden from the user, which makes it hard to study.

This project presents Mini Shell, a Linux shell written in C with a GTK3 graphical front-end. The shell reads a command line, splits it into arguments, and runs the command in a child process created with fork() and execvp(). A command that ends with & is run in the background: the shell prints its job number and process ID, records it in a job table, and returns immediately for the next command. The standard output and standard error of every command are captured through a pipe and shown in a terminal-style output area, and a Background Jobs table shows the job ID, PID, command, and status of every running background job.

To keep the window responsive, the program does not block. Command output is read through a GLib file-descriptor watch, and finished children are collected by a timer that calls waitpid() with WNOHANG every 100 ms, so that no zombie processes remain. The built-in commands jobs, kill, cd, clear, and exit are provided, jobs can be terminated by command or with a button, and all remaining children are terminated when the window is closed.

The report documents the requirements, design, implementation, testing approach, limitations, and user manual of the system. Future extensions include pipelines and redirection, quoting, foreground job control, escalation from SIGTERM to SIGKILL, and support for interactive programs through a pseudo-terminal.

Contents

Chapter 1 Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Motivation 1
1.4 Objectives 2
1.5 Scope 2

Chapter 2 Review of Literature **3**
2.1 Overview 3
2.2 The Unix Process Model 3
2.3 Waiting for Children and Zombie Processes 3
2.4 Shell Job Control and the Background Operator 3
2.5 Pipes and Output Capture 3
2.6 GTK3 and the GLib Main Loop 4
2.7 Technology Comparison 4
2.8 Research Gap and Project Position 4
2.9 References Considered 4

Chapter 3 Project Vision **5**
3.1 Problem Statement 5
3.2 Business Opportunity 5
3.3 Objectives 5
3.4 Project Scope 5
3.5 Constraints 5
3.6 Stakeholders Description 6
3.7 Stakeholders Summary 6
3.8 Key High Level Goals and Problems 6

Chapter 4 Software Requirements Specifications **7**
4.1 List of Features 7
4.2 Functional Requirements 7
4.3 Quality Attributes 8
4.4 Non-Functional Requirements 8
4.5 Use Cases 8
4.6 Sequence Diagram 9
4.7 Test Plan 9
4.8 Software Development Plan 10
4.9 Wire-frames 10
4.10 UI Screens 10

Chapter 5 Iteration Plan **11**

Chapter 6 Iteration 1 ..... **12**
6.1 System Model 12
6.2 Component Diagram 12
6.3 Layer Diagram 12
6.4 Structure Chart 13
6.5 Flow Diagram 13
6.6 Data Dictionary 14
6.7 Activity 14
6.8 State Machine 14
6.9 Algorithm Design 15
6.10 Development Phase 15

6.11 Maintainable Phase 15
Chapter 7 Iteration 2 17
7.1 Iteration Objective 17
7.2 Process Creation 17
7.3 Output Capture 17
7.4 Built-in Commands 18
7.5 Iteration 2 Testing 18
Chapter 8 Iteration 3 19
8.1 Iteration Objective 19
8.2 Software Components 19
8.3 GUI Implementation 19
8.4 Working Principle 19
8.5 Screenshots 20
Chapter 9 Iteration 4 21
9.1 Iteration Objective 21
9.2 Design Challenges and Resolutions 21
9.3 Functional Testing 21
9.4 Testing Procedure 22
9.5 Discussion 22
9.6 Advantages 22
9.7 Limitations 22
Chapter 10 Implementation Details 24
10.1 System Overview 24
10.2 Command Line Parsing 24
10.3 Launching and Output Capture 24
10.4 Monitoring and Reaping 24
10.5 Built-in Commands and Job Termination 24
10.6 Integrated Operation 25
Chapter 11 User Manual 26
11.1 System Setup 26
11.2 Starting the Program 26
11.3 Operating Procedure 26
11.4 Troubleshooting 27
Chapter 12 Conclusions and Future Work 28
12.1 Conclusion 28
12.2 Future Scope 28
12.3 Final Project Perspective 28
Bibliography 29

CHAPTER 1

INTRODUCTION

1.1 Background

A shell is the program through which a user asks an operating system to run other programs. On Linux, every command typed into a shell is carried out by the same small set of system calls: `fork()` creates a new process, `execvp()` replaces that process with the requested program, and `waitpid()` lets the parent collect the result. Understanding these calls is a core part of any operating systems course.

A useful shell also lets the user run long commands in the background by ending the command line with `&`. The shell then returns to the prompt immediately, while the command continues to run. To do this safely the shell must keep a record of every background job, report when a job finishes, allow the user to terminate a job, and make sure that finished child processes are collected so that they do not remain as zombie processes.

This project implements such a shell, called Mini Shell, in the C language. A GTK3 graphical front-end presents a terminal-style output area, a command entry box, and a Background Jobs table that lists every running background job together with its job ID, process ID (PID), command text, and status.

1.2 Problem Statement

Command-line shells hide the process-management work they perform, which makes the underlying concepts hard to observe. The problem addressed by this project is the design of a compact shell that can:

- execute ordinary Linux commands by creating child processes with `fork()` and `execvp()`;
- run commands in the background when the command line ends with `&`;
- track background jobs in a job table and show them live in a graphical list;
- allow a background job to be terminated by command (`kill`) or by a button;
- collect finished children automatically so that no zombie processes remain, while keeping the graphical interface responsive.

1.3 Motivation

Job management is one of the most visible features of a real shell, yet it combines several operating-system ideas in a small amount of code: process creation, inter-process communication through pipes, signals, and non-blocking child collection. Building a working version, and showing its state graphically, gives a clear and testable demonstration of these ideas. The graphical front-end also shows how a blocking, terminal-oriented design must be reorganised to fit an event-driven GUI toolkit.

1.4 Objectives

1. To design and implement a mini shell in C that executes external Linux commands using `fork()`, `execvp()` and `waitpid()`.
2. To support background execution of commands using the trailing `&` operator.
3. To maintain a job table with unique job IDs and to display running background jobs in a GUI table.
4. To capture the standard output and standard error of every command through a pipe and display it in the GUI.
5. To implement the built-in commands `jobs`, `kill`, `cd`, `clear` and `exit`.
6. To reap terminated children without blocking the user interface, so that no zombie processes are left behind.
7. To clean up all running child processes when the application is closed.

1.5 Scope

The scope of the current system includes parsing of a single command line, foreground and background execution of external commands, output capture, a job table of up to 64 entries, five built-in commands, and a GTK3 interface with a Run button and a Kill Selected Job button.

The present system does not include pipelines (`|`), input/output redirection (<, >), quoting or escaping of arguments, environment-variable expansion, wildcards, command history, or interactive programs that need a terminal (for example `vim` or `top`). These are considered future work.

CHAPTER 2

REVIEW OF LITERATURE

2.1 Overview

Shells have been studied and taught since the early days of Unix. Classic textbooks describe the shell as a loop that reads a command, creates a process to run it, and waits for it to finish. Later work on job control added background execution, process groups, and the ability to stop and resume jobs. This chapter summarises the concepts that the present project relies on.

2.2 The Unix Process Model

In Unix-like systems a new process is created by `fork()`, which duplicates the calling process. The child then calls one of the `exec` family of functions, such as `execvp()`, to load a different program. `execvp()` searches the directories in the `PATH` environment variable, so the user can type a command name without its full path. If `exec` fails, the child must terminate itself with `_exit()` so that it does not continue running a copy of the parent.

2.3 Waiting for Children and Zombie Processes

When a child terminates, the kernel keeps a small record of its exit status until the parent collects it with `wait()` or `waitpid()`. A terminated child whose status has not yet been collected is called a zombie. A shell that starts background jobs must therefore collect finished children at suitable moments. The option `WNOHANG` makes `waitpid()` return immediately when the child is still running, which allows the check to be done without blocking.

2.4 Shell Job Control and the Background Operator

In a shell, ending a command line with `&` requests background execution: the shell does not wait for the command and prints a job number and process ID. The shell keeps a job table, and built-in commands such as `jobs` and `kill` operate on that table. Built-in commands are handled inside the shell process itself, because commands such as `cd` must change the state of the shell and cannot be performed by a child process.

2.5 Pipes and Output Capture

A pipe created with `pipe()` provides two file descriptors, one for reading and one for writing. By calling `dup2()` in the child, standard output and standard error can be redirected into the write end, while the parent reads the command output from the read end. This is the mechanism that allows a graphical program to display the output of the commands it launches.

2.6 GTK3 and the GLib Main Loop

GTK3 is a widely used toolkit for graphical programs on Linux. A GTK program is event driven: the call to `gtk_main()` runs the GLib main loop, which dispatches user events, timers, and file-descriptor watches. Long blocking calls inside a callback freeze the window, so a GUI shell must not call a blocking `waitpid()` or `read()` in the middle of an event handler. GLib supplies `g_unix_fd_add()` to be notified when a descriptor is readable, and `g_timeout_add()` to run a function periodically. Tabular data is shown with `GtkTreeView` and `GtkListStore`, and the appearance can be customised with CSS.

2.7 Technology Comparison

Approach	Typical Setting	Main Characteristic
Terminal shell with blocking wait	Console program	Simple; waits for each foreground command, no graphical job view
Terminal shell with SIGCHLD handler	Console program	Reaps children asynchronously; signal handlers must be written carefully
GTK shell with pipe watch and timer polling (this project)	GTK3 desktop application	Responsive window; output and child collection integrated in the main loop; job table shown live

2.8 Research Gap and Project Position

Many teaching shells are console programs that wait for each command, and job management is either missing or hidden. The present project places the same core calls (`fork()`, `execvp()`, `waitpid()`) behind a graphical interface and shows how output capture and child collection can be done safely inside an event loop. It is intended as a clear, small, and readable reference implementation rather than a full replacement for a production shell.

2.9 References Considered

The material considered covers Unix process control, signals and pipes, the manual pages of the system calls used, and the GTK3 and GLib reference documentation. Detailed references are listed in the Bibliography.

CHAPTER 3

PROJECT VISION

3.1 Problem Statement

The project addresses the need for a small, understandable shell that demonstrates process creation and background job management, and that presents running jobs graphically.

3.2 Business Opportunity

The application can be used as a teaching and demonstration tool in operating-systems laboratories, where students can run commands in the background and watch the job table change. The design can also serve as a base for a larger graphical terminal or a job-monitoring utility.

3.3 Objectives

- Execute external commands in the foreground or background.
- Show command output inside the application window.
- Track and display background jobs with job ID, PID, command, and status.
- Terminate background jobs from the keyboard or with a button.
- Validate the design through functional testing.

3.4 Project Scope

- Command parsing by whitespace;
- Process creation with `fork()` and `execvp()`;
- Output capture through `pipe()` and GLib descriptor watches;
- Non-blocking child collection with a 100 ms `waitpid(WNOHANG)` timer;
- Built-in commands `jobs`, `kill`, `cd`, `clear`, `exit`;
- GTK3 user interface written in C.

3.5 Constraints

- The shell supports only simple commands; pipelines, redirection, quoting, and wildcards are not handled.

- The job table is fixed at 64 entries, a command line is limited to 1024 characters, and a command to 63 arguments.
- Commands run without a terminal, so interactive programs that need a terminal do not work.
- The program requires Linux (or another POSIX system), the GTK3 development library, and a graphical display.
- Only one foreground command can run at a time; the entry box is disabled while it runs.

3.6 Stakeholders Description

The stakeholders are the student project team, the project supervisor, faculty members, and users such as students of operating systems who use the tool to learn about processes and jobs. The team is responsible for design, implementation, and testing; the supervisor and faculty provide academic guidance and evaluation.

3.7 Stakeholders Summary

Name	**ID**
[Team Member 1 Name]	[Roll Number]
[Team Member 2 Name]	[Roll Number]
[Team Member 3 Name]	[Roll Number]
[Team Member 4 Name]	[Roll Number]

3.8 Key High Level Goals and Problems

The high-level goal is to provide a graphical mini shell that manages background jobs correctly. The principal problems are keeping the interface responsive while child processes run, capturing their output without losing data, and collecting finished children so that no zombie processes remain.

CHAPTER 4

SOFTWARE REQUIREMENTS SPECIFICATIONS

4.1 List of Features

1. Graphical window with output area, command entry, and Run button.
2. Execution of any external Linux command found in `PATH`.
3. Background execution with the trailing `&` operator.
4. Capture of standard output and standard error of each command.
5. Live Background Jobs table (Job ID, PID, Command, Status).
6. Built-in commands: `jobs`, `kill <job_id>`, `cd [dir]`, `clear`, `exit`.
7. Kill Selected Job button.
8. Automatic reporting of completed or terminated background jobs.
9. Automatic cleanup of all child processes when the window is closed.

4.2 Functional Requirements

ID	Requirement
FR1	The system shall display a window containing an output area, a command entry box, a Run button, and a Background Jobs panel.
FR2	The system shall accept a command line when Enter is pressed or Run is clicked, and echo it in the output area after the prompt `mini-shell>`.
FR3	The system shall split the command line into arguments at spaces and tabs.
FR4	The system shall run external commands in a child process created with `fork()` and `execvp()`.
FR5	The system shall capture the standard output and standard error of the child through a pipe and display them as they are produced.
FR6	The system shall run a command in the background when the line ends with `&`, and print the job ID and PID in the form `[id] pid`.
FR7	The system shall disable the entry box and Run button while a foreground command is running, and enable them again when it finishes.
FR8	The system shall collect finished children without blocking, and report background jobs as Done or Terminated.
FR9	The system shall list running background jobs in the Background Jobs table with Job ID, PID, Command, and Status.

ID	Requirement
FR10	The system shall provide the built-in commands `jobs`, `kill`, `cd`, `clear`, and `exit`.
FR11	The system shall send SIGTERM to the selected job when Kill Selected Job is clicked or `kill <job_id>` is entered.
FR12	The system shall terminate and collect all remaining children when the window is closed.
FR13	The system shall report errors such as command not found, fork or pipe failure, job table full, invalid job ID, and a syntax error near `&`.

4.3 Quality Attributes

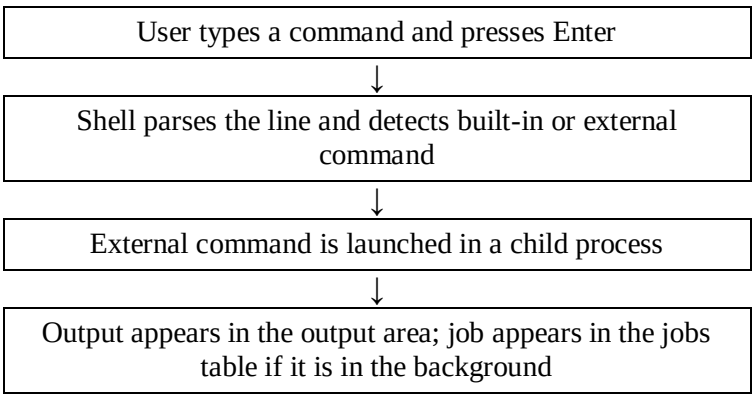
- Responsiveness: the window stays usable while commands run.
- Correctness: no zombie processes and no lost output.
- Simplicity: a single readable C source file.
- Robustness: invalid input and system-call failures produce messages instead of crashes.
- Extensibility: new built-ins can be added in one place.

4.4 Non-Functional Requirements

The program shall build with GCC and `pkg-config` using the warning options `-Wall -Wextra` and shall run on a Linux desktop with GTK 3. Finished background jobs should be reported within about 100 ms, which is the polling interval of the reaping timer. Output text containing invalid UTF-8 shall not crash the interface.

4.5 Use Cases

The main use case is running a command and managing background jobs.



ID	Use Case	Description
UC1	Run foreground command	User enters e.g. `ls -l`; the entry is disabled until the command finishes.
UC2	Run background command	User enters e.g. `sleep 30 &`; job number and PID are printed and the job is listed.
UC3	List jobs	User enters `jobs`; running background jobs are printed.
UC4	Kill job by command	User enters `kill 1` or `kill %1`; SIGTERM is sent to job 1.
UC5	Kill job by button	User selects a row in the table and clicks Kill Selected Job.
UC6	Change directory	User enters `cd <dir>`; the working directory of the shell changes.
UC7	Clear screen	User enters `clear`; the output area is emptied.
UC8	Exit	User enters `exit` or closes the window; all children are terminated.

4.6 Sequence Diagram

User / GUI	Shell Core (parent)	Child Process
Enter command line →	Parse; create pipe; fork()	
	Close write end; register pipe watch	dup2 pipe to stdout/stderr; execvp()
← Output text shown	read() from pipe in main loop	Program writes output
	Timer: waitpid(WNOHANG)	Program exits
← Done / Terminated message; table updated	Mark reaped; free slot after pipe EOF	

4.7 Test Plan

Test	Purpose	Expected Observation
Foreground command	Verify fork/exec and output capture	Output of `ls` or `echo hello` appears; entry re-enabled afterwards
Background command	Verify `&` handling	`[1] <pid>` printed; job listed with status Running
Reaping	Verify automatic collection	`[1] Done (PID n) cmd` printed when the job ends; row removed

Test	Purpose	Expected Observation
Kill	Verify termination	`Terminated` reported; row removed
Error handling	Verify messages	`command not found` and usage messages appear
Exit	Verify cleanup	No child processes remain after the window is closed

4.8 Software Development Plan

1. Define the job table and data structures.
2. Implement command parsing and the launch function with `fork()` and `execvp()`.
3. Add pipe-based output capture and the descriptor watch.
4. Add the reaping timer and job-table refresh.
5. Implement the built-in commands.
6. Build the GTK3 window and connect the callbacks.
7. Perform functional testing and fix defects.

4.9 Wire-frames

The window is organised from top to bottom as shown below.

Output area – dark, terminal-style text view (read-only, scrolls automatically)
`mini-shell>` [command entry box] [Run]
Background Jobs frame: table with columns Job ID | PID | Command | Status
[Kill Selected Job] (right aligned)

Figure 4.1: Wire-frame of the Mini Shell window

4.10 UI Screens

The user interface consists of the main window titled “Mini Shell - Background Job Manager”, with a default size of 760 × 560 pixels. The output area uses a dark background (#1e1e1e) with light monospace text, and the prompt label is shown in bold green. A welcome message listing the supported commands is displayed when the program starts.

[Insert screenshot of the main window here]

Figure 4.2: Main window of Mini Shell

CHAPTER 5

ITERATION PLAN

The project is organised into four documentation iterations.

Iteration	Focus	Activities
1	System Design	Architecture, components, data structures, algorithms
2	Core Shell Engine	Parsing, fork/exec, pipe capture, built-in commands
3	GUI Integration	GTK3 window, job table, event-loop integration
4	Testing	Functional testing, issue resolution, results

CHAPTER 6

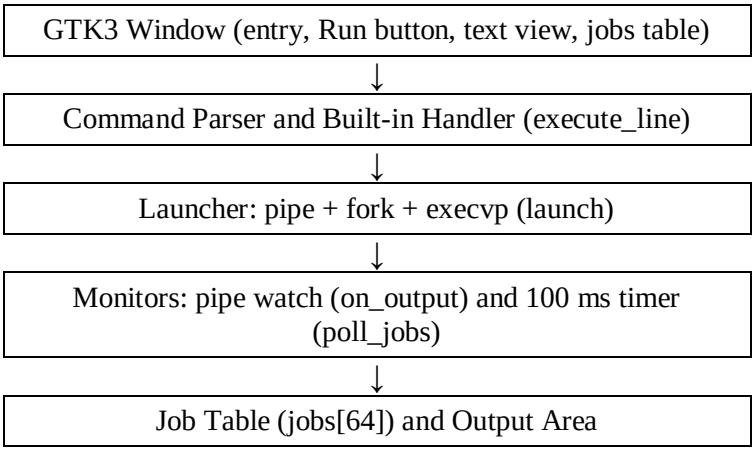
ITERATION 1

6.1 System Model

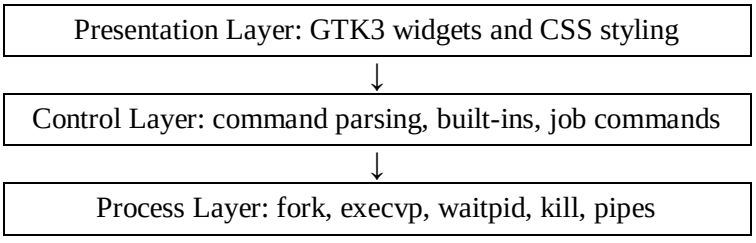
The system consists of four functional areas:

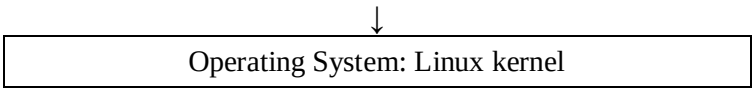
1. **Input and parsing:** the entry box supplies a command line, which is trimmed, checked for a trailing `&`, and split into arguments.
2. **Execution engine:** external commands are started with `fork()` and `execvp()`, with standard output and standard error redirected to a pipe.
3. **Job manager:** a fixed-size job table records each child, its PID, command text, and state; a timer collects finished children.
4. **Presentation:** a GTK3 text view shows command output and messages, and a tree view shows the running background jobs.

6.2 Component Diagram



6.3 Layer Diagram



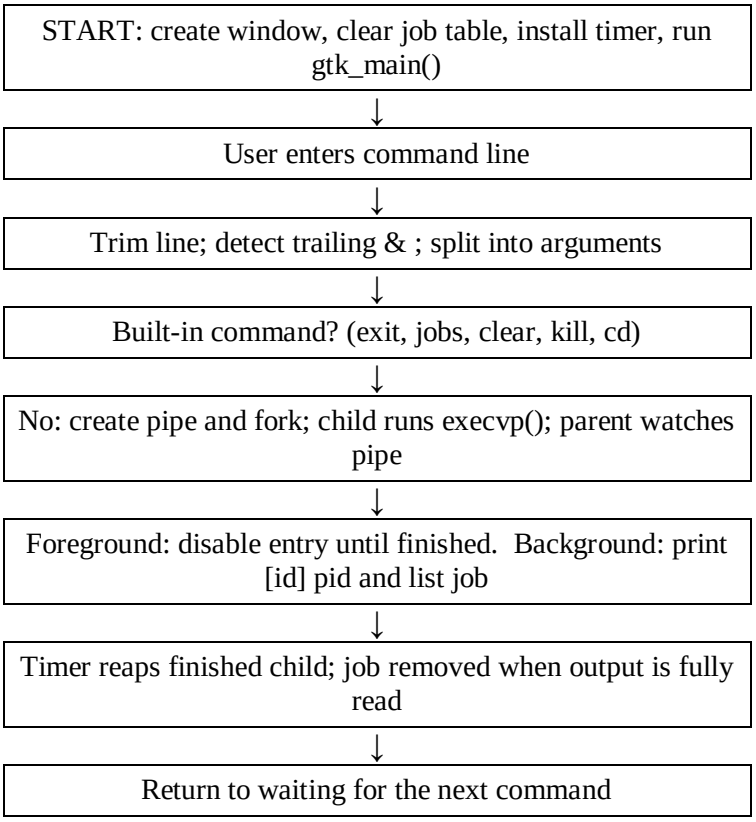


6.4 Structure Chart

Module	Functions
Output helpers	append_text(); refresh_job_list(); set_busy()
Command processing	execute_line(); launch()
Built-in commands	do_jobs(); do_kill(); kill_all_jobs(); cd and clear handled in execute_line()
Monitoring	on_output() (pipe data); poll_jobs() (timer, reaping)
GUI callbacks	on_run(); on_kill_clicked(); on_destroy()
Start-up	main(); add_column()

Figure 6.1: System structure (modules)

6.5 Flow Diagram



6.6 Data Dictionary

Data Element	Type	Purpose
Job.used	int	Marks a job-table slot as occupied
Job.id	int	Job number for background jobs (0 for foreground commands)
Job.pid	pid_t	Process ID of the child
Job.cmd	char[1024]	Command text shown in the table and messages
Job.background	int	1 if started with &
Job.reaped	int	1 once waitpid() has collected the child
Job.killed	int	1 if the user requested termination (reported as Terminated)
Job.out_fd	int	Read end of the output pipe (-1 when closed)
jobs[MAX_JOBS]	Job array	The job table (64 entries)
next_job_id	int	Counter used to give background jobs increasing IDs
fg_running	int	Set while a foreground command is running
MAX_LINE / MAX_ARGS	constants	1024 characters per line; 64 argument slots (63 arguments plus NULL)

6.7 Activity

The activity sequence is: read the command line, trim and check for `&`, split into arguments, handle a built-in or create a pipe, fork, run the program in the child, watch the pipe in the parent, display output, collect the child with the timer, update the job table, and wait for the next command.

6.8 State Machine

Each job passes through the following states. A slot is **Free** until `launch()` fills it and the job becomes **Running**. When the timer finds that the child has ended, the job is marked **Reaped**; background jobs print Done or Terminated at this moment. The slot returns to **Free** only when the job is reaped and the output pipe has reached end-of-file, so that no output is lost.

At application level the shell alternates between **Idle** (entry enabled) and **Busy** (a foreground command is running and the entry and Run button are disabled). Busy returns to Idle when the foreground job has been reaped and its output has been fully read.

6.9 Algorithm Design

The line is parsed as follows. Leading and trailing white space is removed; if the last character is `&`, the background flag is set and the `&` is removed; if nothing remains, a syntax error is reported. The remaining text is split on spaces and tabs with `strtok()`.

The reaping algorithm runs every 100 ms:

```
for each used job j:
    if not j.reaped:
        if waitpid(j.pid, WNOHANG) == j.pid:
            j.reaped = 1
            if j.background: print Done/Terminated message
    if j.reaped and j.out_fd < 0:
        if not j.background: set_busy(0)
        j.used = 0                        /* free the slot */
if any job changed: refresh the jobs table
```

6.10 Development Phase

6.10.1 Unit Test

Component-level tests cover command parsing, the launcher, output capture, each built-in command, the reaping timer, and the cleanup performed when the window is closed.

6.10.2 Test Cases

ID	Test Case	Expected Result
TC1	`echo hello`	hello appears in the output area
TC2	`sleep 30 &`	`[1] <pid>` printed; row added to the table
TC3	`jobs`	Running background jobs listed, or “No running background jobs.”
TC4	`kill 1` / `kill %1`	SIGTERM sent; job reported Terminated
TC5	`kill abc`	Message: ‘abc’ is not a valid job ID
TC6	`nosuchcmd`	Message: command not found
TC7	`&`	Message: syntax error near ‘&’
TC8	Close window with jobs running	All children terminated; no orphans left

6.11 Maintainable Phase

The project is a standalone desktop program contained in a single source file, `mini_shell_gui.c`. Maintainability is supported by the small number of functions, each with a single responsibility, and by comments at the start of every section. A CI/CD pipeline and a deployment diagram are not required for the current prototype.

6.11.1 Configuration and Setup

- Linux system with a graphical desktop (X11 or Wayland);
- GCC compiler and `build-essential`;
- `pkg-config` and the GTK3 development package (`libgtk-3-dev`);
- Source file `mini_shell_gui.c`.

```
sudo apt install build-essential pkg-config libgtk-3-dev
gcc -Wall -Wextra -o mini_shell_gui mini_shell_gui.c \
    $(pkg-config --cflags --libs gtk+-3.0)
./mini_shell_gui
```

CHAPTER 7

ITERATION 2

7.1 Iteration Objective

The second iteration covers the core shell engine: command parsing, process creation, output capture through a pipe, and the built-in commands.

7.2 Process Creation

`launch()` first finds a free slot in the job table, then creates a pipe and calls `fork()`. In the child, the write end of the pipe is duplicated onto standard output and standard error, standard input is redirected to `/dev/null`, and `execvp()` is called. If `execvp()` fails, a message is written to the pipe and the child ends with `_exit(127)`.

```
if (pid == 0) {                                 /* ---- CHILD ---- */
    close(p[0]);
    dup2(p[1], STDOUT_FILENO);                  /* stdout -> pipe */
    dup2(p[1], STDERR_FILENO);                  /* stderr -> pipe */
    close(p[1]);
    int devnull = open("/dev/null", O_RDONLY);
    if (devnull >= 0) { dup2(devnull, STDIN_FILENO); close(devnull); }
    execvp(argv[0], argv);
    ...                                         /* only reached on failure */
    _exit(127);
}
```

In the parent, the write end is closed, the job entry is filled in, and the read end is registered with the GLib main loop. A background job receives the next job ID and the line `[id] pid` is printed; a foreground command switches the interface to the busy state.

7.3 Output Capture

`g_unix_fd_add()` registers the function `on_output()` for the read end of the pipe. Whenever data is available, up to 1023 bytes are read and appended to the output area. When `read()` returns zero (end-of-file) the descriptor is closed and the watch is removed. Because standard error is joined to the same pipe, error messages from commands appear in order with normal output.

7.4 Built-in Commands

Command	Behaviour
`jobs`	Prints each running background job as `[id] Running PID=n command`, or "No running background jobs."
`kill <id>` or `kill %<id>`	Validates the number with `strtol()`, finds the running job, sets its killed flag, and sends SIGTERM.
`cd [dir]`	Calls `chdir()`; with no argument it uses the `HOME` environment variable.
`clear`	Empties the output text buffer.
`exit`	Destroys the window, which runs the cleanup handler.

7.5 Iteration 2 Testing

The core engine was designed so that each part can be exercised by typing commands into the window: foreground commands (TC1), background commands (TC2), the job listing (TC3), termination (TC4, TC5), and error cases (TC6, TC7). The results are recorded in Chapter 9.

CHAPTER 8

ITERATION 3

8.1 Iteration Objective

The third iteration builds the graphical interface and integrates it with the shell engine through the GLib main loop, so that the window stays responsive while commands run.

8.2 Software Components

Component	Qty.	Purpose
GtkWindow (760 × 560)	1	Main application window
GtkTextView in GtkScrolledWindow	1	Read-only terminal-style output area with auto-scroll
GtkEntry and GtkLabel (prompt)	1 + 1	Command input with the `mini-shell>` prompt
GtkButton “Run”	1	Runs the command in the entry box
GtkTreeView with GtkListStore	1	Background Jobs table with four columns
GtkButton “Kill Selected Job”	1	Terminates the selected job
GtkCssProvider	1	Dark terminal look, monospace font, green prompt

8.3 GUI Implementation

The window contains a vertical box with three parts: the scrolled output area, the command row (prompt, entry, Run button), and a “Background Jobs” frame holding the table and the Kill button. The list store has four columns (Job ID, PID, Command, Status); `refresh_job_list()` clears and refills it from the job table whenever a background job starts or ends. Pressing Enter in the entry and clicking Run both call `on_run()`.

8.4 Working Principle

The program never blocks inside a callback. Command output arrives through the descriptor watch, and child termination is detected by the timer that calls `waitpid()` with `WNOHANG` every 100 ms. A timer was chosen instead of a SIGCHLD signal handler because it is simple and safe to combine with GTK, which is not designed for work inside asynchronous signal handlers. While a foreground command is running, `set_busy(1)` disables the entry and Run button; when the timer has reaped the child and the pipe is closed, `set_busy(0)` re-enables them and returns focus to the entry.

CHAPTER 9

ITERATION 4

9.1 Iteration Objective

The fourth iteration covers functional testing, the design problems that had to be solved, and discussion of the results.

9.2 Design Challenges and Resolutions

Challenge	Risk	Resolution in the Program
Window freezing	A blocking read() or waitpid() would stop the GUI	Output is read through `g_unix_fd_add()`; children are checked with a non-blocking timer
Zombie processes	Finished children stay in the process table	`poll_jobs()` calls `waitpid(WNOHANG)` every 100 ms for every job
Lost output	Freeing a job early could drop unread data	A slot is freed only when the job is reaped and its pipe is at end-of-file
Broken pipe	Writing to a closed pipe would kill the process	`SIGPIPE` is ignored with `signal(SIGPIPE, SIG_IGN)`
Child reading the keyboard	A child waiting for input would hang a foreground job	Standard input of the child is redirected to `/dev/null`
Invalid text	Binary output can break the GTK text buffer	`g_utf8_make_valid()` is applied before inserting text
Command not found	The child would continue as a copy of the shell	A message is printed and the child ends with `_exit(127)`
Leftover children	Jobs would keep running after the window is closed	`on_destroy()` calls `kill_all_jobs()` before quitting

9.3 Functional Testing

The table below is to be completed after running each test on the target machine.

Component	Test Purpose	Observed Result
Foreground execution	Verify fork/exec and output	[Enter observed result]
Background execution	Verify `&` and job listing	[Enter observed result]
Reaping	Verify Done message and no zombies	[Enter observed result]

Component	Test Purpose	Observed Result
Kill command and button	Verify termination	[Enter observed result]
Error handling	Verify messages	[Enter observed result]
Cleanup on exit	Verify no orphan children	[Enter observed result]

9.4 Testing Procedure

1. Compile the program and start `./mini_shell_gui`.
2. Run `echo hello` and `ls -l` and check the output.
3. Run `sleep 30 &` and `sleep 60 &`; check the printed job lines and the table.
4. Run `jobs` and compare it with the table.
5. Run `kill 1`, then select the second job and click Kill Selected Job.
6. In another terminal run `ps -ef | grep sleep` or `ps aux | grep defunct` to check that no zombie remains.
7. Run an unknown command, `kill abc`, and a lone `&` to check the error messages.
8. Start a background job and close the window; confirm with `ps` that the job has been terminated.

9.5 Discussion

The design shows that a shell with background job management can be placed inside an event-driven GUI without threads. The main points are to hand all blocking work to the kernel, to read output only when the descriptor is ready, and to detect child termination by polling. Freeing a job slot only after both conditions (child reaped and pipe closed) are met keeps the output complete even for commands that finish very quickly.

9.6 Advantages

- Small, readable, single-file implementation.
- Responsive window; output appears while a command is running.
- Live job table and completion messages.
- No zombie processes and clean shutdown.
- Safe parsing with bounds on line, argument, and job counts.

9.7 Limitations

- No pipelines, redirection, quoting, wildcards, or variable expansion.
- A foreground command cannot be interrupted from the interface; the entry stays disabled until it finishes.
- `kill` sends only SIGTERM to the child process; a program that ignores it keeps running, and there is no escalation to SIGKILL.
- `kill_all_jobs()` waits for each child at exit, so a program that ignores SIGTERM could delay closing the window.
- Commands run without a terminal, so full-screen or interactive programs such as `vim` and `top` are not supported.
- Fixed limits: 1024 characters per line, 63 arguments, 64 jobs.
- Job IDs only increase and are not reused; the output text buffer is not trimmed.
- Completion is detected by a 100 ms timer, so messages can appear up to 100 ms late.

CHAPTER 10

IMPLEMENTATION DETAILS

10.1 System Overview

The program is written in C with GTK3 and consists of the single source file `mini_shell_gui.c`. It uses the POSIX headers `unistd.h`, `fcntl.h`, `signal.h`, `sys/wait.h` and the GTK/GLib headers `gtk/gtk.h` and `glib-unix.h`. The macro `_POSIX_C_SOURCE 200809L` is defined to expose the required POSIX functions.

10.2 Command Line Parsing

`execute_line()` copies the input, strips white space with `g_strstrip()`, echoes the line, and checks for a trailing `&`. The text before the `&` is kept as the command text for the job table. The line is then tokenised with `strtok()` using a space and a tab as separators, up to `MAX_ARGS - 1` tokens, and the argument array is terminated with NULL. The first token decides whether the command is a built-in or an external program.

10.3 Launching and Output Capture

For external commands, `launch()` selects a free job slot, creates the pipe, and forks as described in Chapter 7. The parent records the job (`used`, `pid`, `background`, `out_fd`, `id`, `cmd`) and registers `on_output()` with `g_unix_fd_add()`. Foreground jobs get ID 0 and are not shown in the table.

10.4 Monitoring and Reaping

`poll_jobs()` is installed with `g_timeout_add(100, poll_jobs, NULL)`. For every unreaped job it calls `waitpid(pid, &status, WNOHANG)`. When the child has ended, the job is marked reaped and, for background jobs, a message such as `[1] Done (PID 4321) sleep 5` is printed ("Terminated" if the user killed it). The slot is released when the job is reaped and `out_fd` is closed, and the jobs table is refreshed if anything changed.

10.5 Built-in Commands and Job Termination

`do_jobs()` walks the job table and prints running background jobs. `do_kill()` finds the running background job with the given ID, sets `killed`, and sends SIGTERM with `kill()`.

The same function is used by the `kill` command and by the Kill Selected Job button, which reads the Job ID from the selected row of the tree view. At exit, `kill_all_jobs()` sends SIGTERM to every unreaped child and waits for it.

10.6 Integrated Operation

At start-up `main()` initialises GTK, ignores SIGPIPE, sets every `out_fd` to -1, installs the CSS style, builds the window and the widgets, prints the welcome message, installs the 100 ms timer, shows the window, and enters `gtk_main()`. From then on, all work is done in callbacks: `on_run()` for input, `on_output()` for command output, `poll_jobs()` for child collection, `on_kill_clicked()` for the Kill button, and `on_destroy()` for shutdown.

CHAPTER 11

USER MANUAL

11.1 System Setup

Install the compiler and the GTK3 development files once, then compile the program:

```
sudo apt install build-essential pkg-config libgtk-3-dev
gcc -Wall -Wextra -o mini_shell_gui mini_shell_gui.c \
    $(pkg-config --cflags --libs gtk+-3.0)
```

11.2 Starting the Program

Run `./mini_shell_gui` from a terminal inside a graphical desktop session. The window opens with a welcome message and the cursor in the command box.

11.3 Operating Procedure

1. Type a command in the entry box and press Enter or click Run.
2. To run a command in the background, end it with `&`, for example `sleep 30 &`.
3. Watch the Background Jobs table for the job ID, PID, command, and status.
4. Type `jobs` to list running background jobs in the output area.
5. To stop a job, type `kill <job_id>` (or `kill %<job_id>`), or select its row and click Kill Selected Job.
6. Use `cd <directory>` to change the working directory, `clear` to empty the output area, and `exit` or the window close button to quit.

Command	Effect
`ls -l`	Runs in the foreground; the entry is disabled until it ends
`sleep 30 &`	Starts a background job and prints `[1] <pid>`
`jobs`	Lists running background jobs
`kill 1`	Sends SIGTERM to job 1
`cd /tmp`	Changes the current directory
`clear`	Clears the output area
`exit`	Terminates all jobs and closes the program

11.4 Troubleshooting

Problem	Possible Check
Compile error: gtk/gtk.h or gtk+-3.0 not found	Install `libgtk-3-dev` and `pkg-config`
Window does not open	A graphical session (DISPLAY) is required; it will not run on a plain text console
Entry box is greyed out	A foreground command is still running; wait for it to finish (use `&` for long commands)
“command not found”	Check the spelling and that the program is in `PATH`
“Select a job in the list first.”	Click a row in the Background Jobs table before pressing Kill
Job remains after kill	The program may ignore SIGTERM; stop it from another terminal with `kill -9 <pid>`
“job table full”	Wait for or kill running jobs (limit is 64)

CHAPTER 12

CONCLUSIONS AND FUTURE WORK

12.1 Conclusion

The project implements a mini Linux shell with background job management and a GTK3 graphical front-end. It runs external commands with `fork()`, `execvp()` and `waitpid()`, captures their output through pipes, supports background execution with `&`, maintains a live job table, provides the built-ins `jobs`, `kill`, `cd`, `clear` and `exit`, and collects finished children without blocking the interface or leaving zombie processes.

12.2 Future Scope

1. Pipelines and input/output redirection.
2. Quoting, escaping, wildcards, and environment-variable expansion.
3. Foreground interruption (Ctrl+C) and `fg`, `bg` and stop/continue support using process groups and SIGSTOP/SIGCONT.
4. Escalation from SIGTERM to SIGKILL, and signalling the whole process group.
5. A pseudo-terminal so that interactive programs can run.
6. Command history and tab completion.
7. A limit on the size of the output buffer and separate output per job.

12.3 Final Project Perspective

The project shows that the essential parts of a shell can be implemented clearly in a few hundred lines of C, and that these parts fit naturally into an event-driven graphical program. The modular structure allows parsing, execution, job management, and the interface to be improved independently, which provides a good base for a more complete shell.

Bibliography

[1] W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed., Addison-Wesley, 2013.

[2] M. Kerrisk, The Linux Programming Interface, No Starch Press, 2010.

[3] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, Wiley.

[4] Linux man-pages: fork(2), execvp(3), waitpid(2), pipe(2), dup2(2), kill(2), signal(7).

[5] The GTK Project, GTK 3 Reference Manual.

[6] The GNOME Project, GLib Reference Manual (g_unix_fd_add, g_timeout_add).